

Fundsroom Mini ERP + CRM Operations Portal

Technical Project Documentation

Version: 1.0.0

Date: August 12, 2026

Developer / Repository: Ankushsph/fundsroom

Technology Stack: Node.js, TypeScript, Express, React, Vite, Tailwind CSS, Prisma ORM, PostgreSQL

Table of Contents

1. Executive Summary
2. Problem Statement
3. Objectives
4. Functional Modules
5. Technology Stack
6. System Architecture
7. Database Design & Entity Relationship Diagram
8. Authentication & Role-Based Access Control (RBAC)
9. Customer CRM
10. Product Catalog Management
11. Inventory & Stock Management
12. Sales Challan Management
13. Atomic Challan Confirmation
14. API Documentation
15. Frontend Client Application
16. Security Infrastructure
17. Testing & Verification
18. Deployment Details
19. User Roles & Permission Matrix
20. User Guide
21. Deployment & Operational Notes
22. Limitations & Future Scope
23. Conclusion

1. Executive Summary

The **Fundsroom Mini ERP + CRM Operations Portal** is an enterprise-grade, full-stack application designed to streamline business operations for wholesale and distribution companies. By consolidating Customer Relationship Management (CRM) and Enterprise Resource Planning (ERP) workflows into a single system, it bridges the gap between sales agents, warehouse administrators, finance accounts teams, and system administrators.

At its core, the system coordinates customer lifecycle tracking, product catalogs, real-time inventory adjustments, and sales challan lifecycles. It utilizes robust transactional operations to manage stock allocations, ensuring high consistency during high-concurrency periods.

2. Problem Statement

Wholesale and distribution businesses operate in fast-paced environments where sales agents and warehouse workers frequently perform concurrent actions.

Traditional operations face the following challenges: * **Over-allocation and**

Negative Stock: Multiple sales agents confirming orders concurrently can exceed available inventory levels. This leads to double-allocation, shipping delays, and

system inaccuracies. * **Lack of Role Segregation:** Sensitive operational data—such as product cost margins or customer status—can be edited by unauthorized users. *

Inefficient Stock Auditing: Without automated logs for stock movements (Stock IN/OUT), business owners cannot trace inventory changes, leading to shrinkage. *

Data Inconsistency: Network failures or concurrent edits can create orphaned entries in the database, resulting in mismatch between invoices, challans, and stock levels.

3. Objectives

The system addresses these operational bottlenecks with the following objectives: 1.

Concurrency Safety: Implement database-level transaction mechanisms to guarantee that stock cannot drop below zero, even under heavy parallel request loads. 2. **Comprehensive Role-Based Access Control:** Separate administrative, sales,

inventory, and accounting functions using strict middleware authorization checks. 3.

Auditability: Track all inventory events (Stock IN, Stock OUT, challan deduction, and cancellations) in a permanent ledger. 4. **Historical Pricing Snapshots:** Record prices and product details at the exact moment a transaction occurs, protecting history

from future updates. 5. **Modern, Responsive SPA:** Deliver a high-performance React-based single-page application with immediate client-side feedback and

validation.

4. Functional Modules

The application is organized into six core functional modules:

- * **Authentication:** Handles user validation using hashed passwords, issues JSON Web Tokens (JWT) for secure requests, and provides session verification.
- * **Role-Based Access Control (RBAC):** A middleware layer that checks a user's role against endpoint requirements, enforcing permission limits.
- * **Customer CRM:** Manages customer leads, wholesale clients, contact information, GST registry verification, and timeline-based notes.
- * **Product Management:** A centralized catalog for tracking SKUs, pricing, categories, storage locations, and alert thresholds.
- * **Inventory & Stock Movements:** Tracks stock levels via direct operations (Stock IN/OUT) and automatically logs changes to the ledger.
- * **Sales Challans:** Handles draft challan creation, line item calculations, automatic status transitions, and atomic database confirmation.

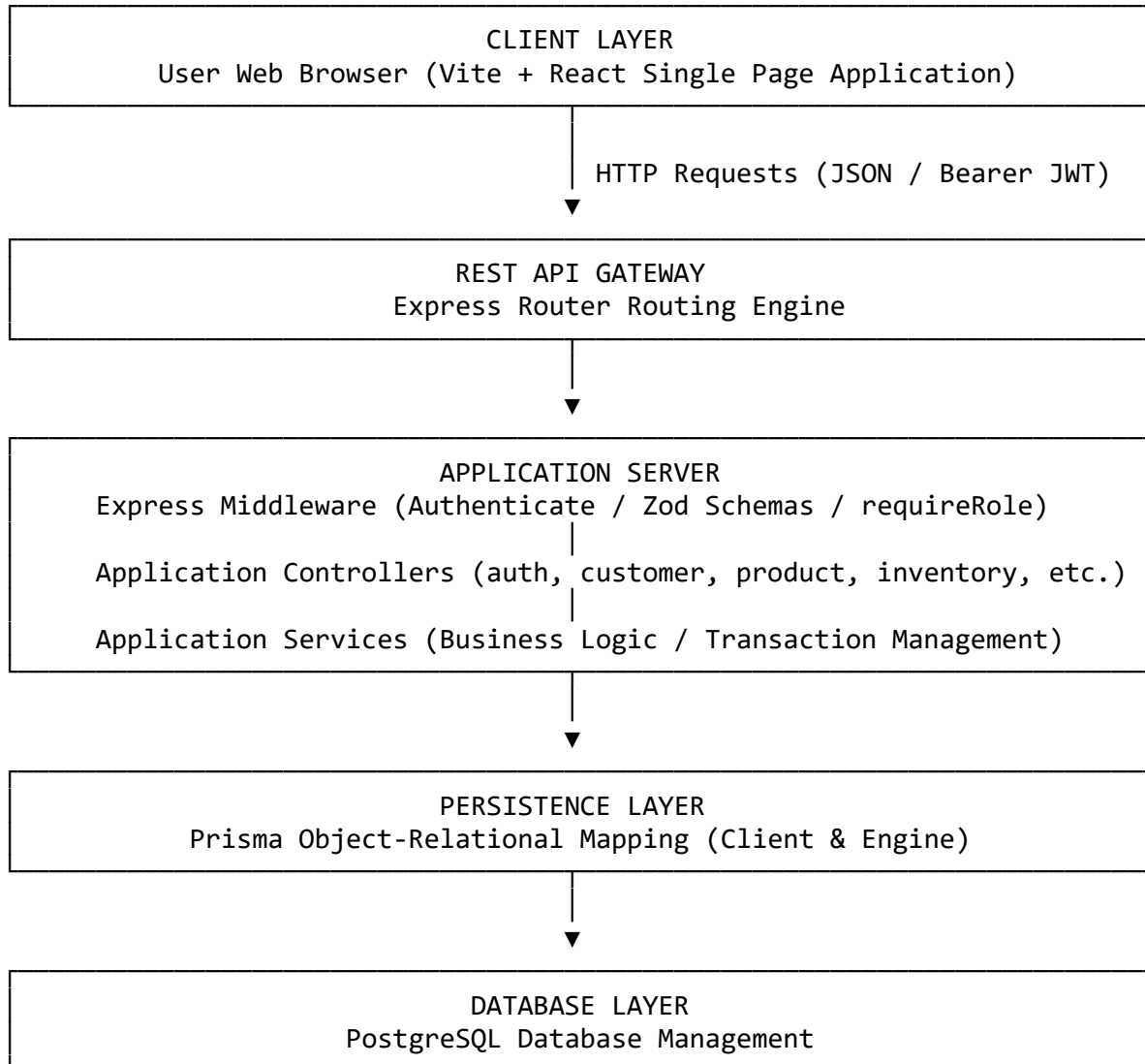
5. Technology Stack

The table below lists the libraries and tools used in both the backend and frontend services:

Component	Technology	Version	Purpose / Detail
Frontend Runtime	React	^18.2.0	User interface rendering
Frontend Build Tool	Vite	^5.0.8	Fast asset bundling and dev server
Styling	Tailwind CSS	^3.4.1	Utility-first CSS styling framework
API Client	Axios	^1.6.5	HTTP request client with interceptors
Routing	React Router DOM	^6.21.0	Client-side routing and layout management
Backend Runtime	Node.js / Express	Node 24 / ^4.18.2	REST API hosting environment
Language	TypeScript	^5.3.3	Strict compiler-enforced static typing
Database ORM	Prisma	^5.22.0	Schema modeling and database client generation
Database	PostgreSQL	18 (Render)	Persistent database engine
Validation	Zod	^3.22.4	Request payload and data schema validation
Security / Hashing	Bcrypt	^5.1.1	Password hashing algorithm
Token Auth	JsonWebToken	^9.0.2	JWT issuance, signing, and verification
Process Manager	ts-node-dev	^2.0.0	Hot-reload development engine

6. System Architecture

The application follows a standard layered architecture with strict separation between UI rendering, routing, middleware, business services, and database layers:

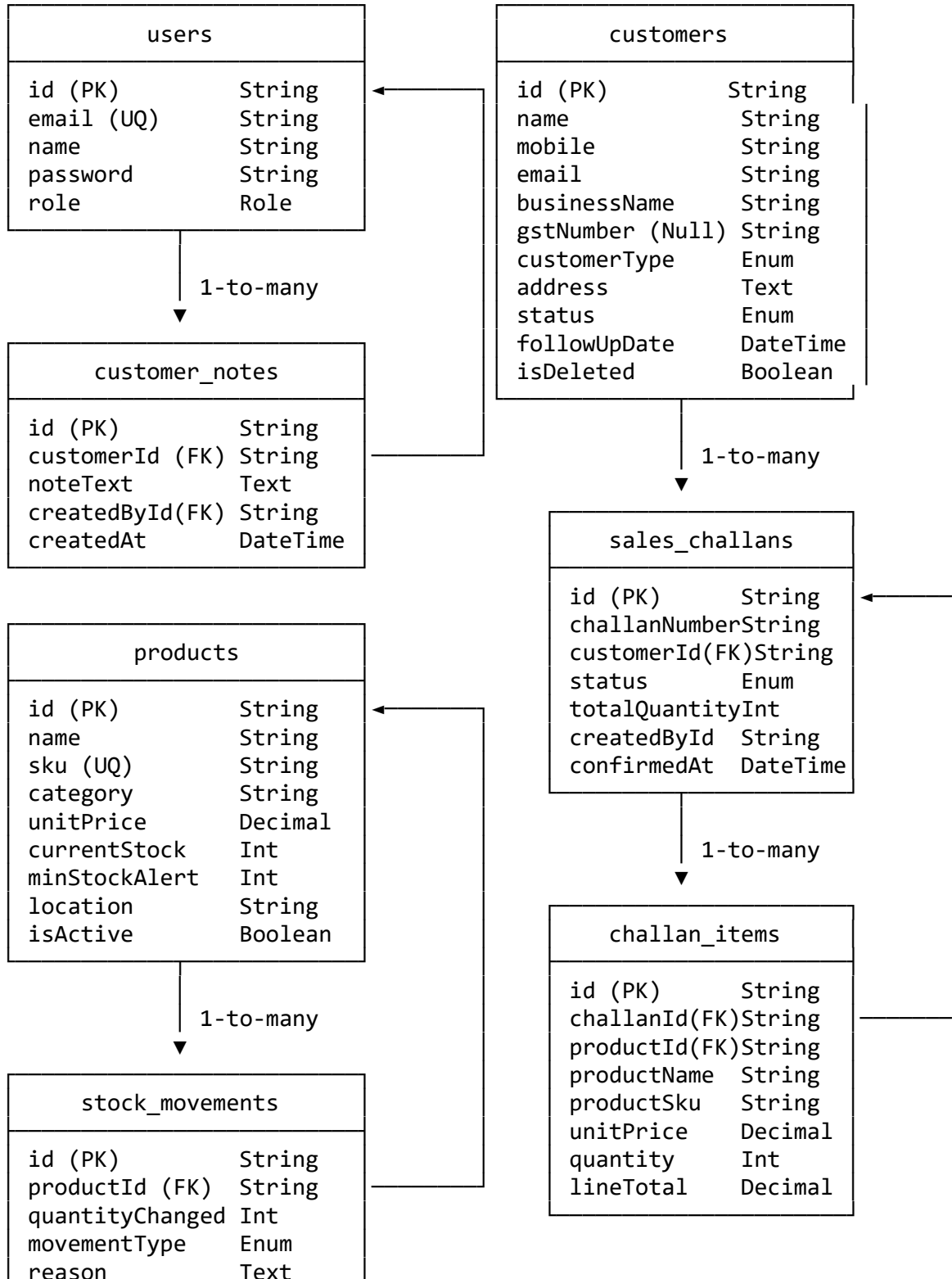


The system uses standard HTTP requests to communicate. Express acts as the router, running incoming requests through an authentication middleware (to extract and verify JWT tokens) and authorization middleware (to check role-based permissions).

Once authorized, the controllers call corresponding services that implement the business logic. Database updates are handled through Prisma, utilizing transactions when atomic operations are required.

7. Database Design & ERD

The database consists of 7 relational tables defined in the Prisma schema. These models capture the operational states of the portal:



createdById(FK) String

Model Schemata Details

- **User (users):** Represents system employees. Includes a unique index on email and a 1-to-many relationship with note creations, stock movements, and sales challans.
- **Customer (customers):** Tracks accounts. Includes soft deletion (isDeleted flag) and indexes on name, email, mobile, status, and customerType.
- **CustomerNote (customer_notes):** Stores historical follow-ups. Has a Cascade delete relation to Customer and a link to the creator User.
- **Product (products):** Tracks inventoried catalog items. Includes a unique index on sku, minStockAlert thresholds, and a location coordinate.
- **StockMovement (stock_movements):** Audits stock changes. Automatically logs quantity changes, movement type (IN/OUT), and reasons.
- **SalesChallan (sales_challans):** Represents purchase orders. Features a unique challanNumber, status (DRAFT, CONFIRMED, CANCELLED), and total calculations.
- **ChallanItem (challan_items):** Tracks order line items. Captures immutable historical snapshots (productName, productSku, unitPrice at checkout time) to prevent future product changes from altering past invoices.

8. Authentication & RBAC

The system uses a stateless token architecture based on JSON Web Tokens:

- Login Validation:** The user provides their email and password. The system retrieves the matching record, compares the password using `bcrypt.compare`, and returns a signed token if valid.
- Token Generation:** Tokens are signed with a server-side secret (`JWT_SECRET`) and contain the following payload claims: `json { "userId": "usr_c123456789", "role": "SALES" }`
- Verification:** Protected routes require an Authorization header formatted as `Bearer <token>`. The `authenticate` middleware decodes this token and attaches the user profile to the Express request object (`req.user`).
- Role Check (RBAC):** The `requireRole` middleware checks if the user's role matches the allowed roles. If unauthorized, it returns a 403 Forbidden error. Unauthenticated requests return a 401 Unauthorized error.

9. Customer CRM

The Customer CRM module handles account tracking and logging follow-up interactions:

- * **Customer CRUD:** Sales and Admin users can create and edit profiles. Only Admins can soft-delete a customer (`isDeleted: true`).
- * **Input Validation:** `createCustomerSchema` requires a valid email format, 10-15 digit mobile numbers, and optionally checks for a 15-character alphanumeric GST number.
- * **Customer Notes:** Provides a timeline for tracking interactions. Sales agents can add notes, but can only delete their own. Admins have full access to delete any note.

10. Product Catalog Management

The Product Catalog module manages items in the company's inventory:

- * **Product CRUD:** Admins and Warehouse users can create and update catalog products. SKU values must be unique and are immutable once created.
- * **Product Deactivation:** To preserve transaction history, products are soft-deleted by setting `isActive: false` instead of being removed from the database.
- * **Price Decimals:** Prices are stored using the PostgreSQL `Decimal(10,2)` type, avoiding the floating-point rounding errors typical of float values.

11. Inventory & Stock Management

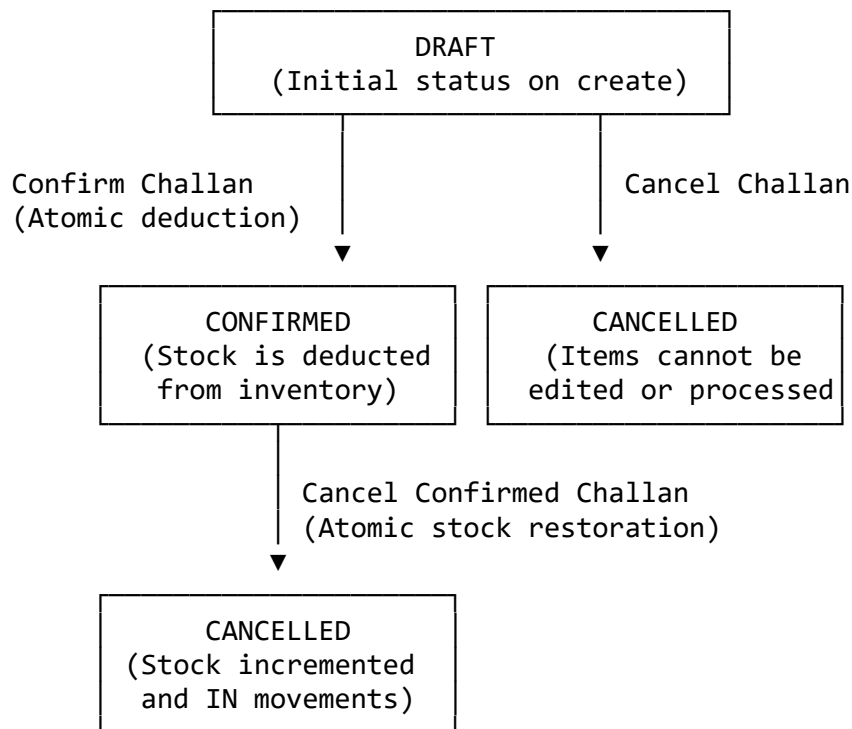
The Inventory module monitors stock levels and logs changes:

- * **Direct Stock Adjustments:** Admins and Warehouse staff can manually trigger Stock IN and Stock OUT events. Quantity inputs must be integers greater than zero.
- * **Transaction Logging:** Manual adjustments run inside a database transaction, which updates the product's `currentStock` and writes a new audit entry to `stock_movements`.

Negative Stock Prevention: Stock OUT requests are verified at the database level to ensure the deducted quantity does not exceed the available stock.

12. Sales Challan Management

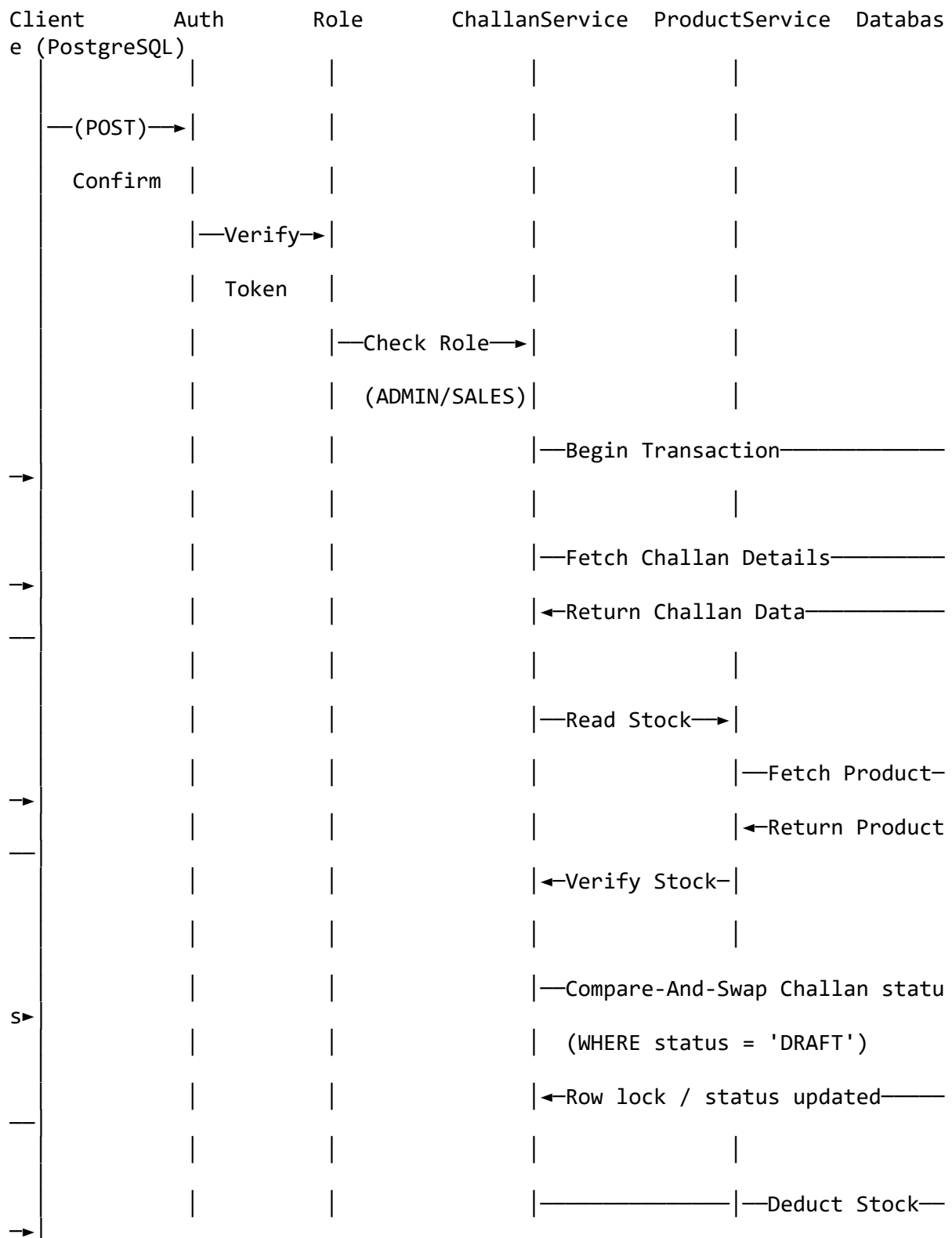
The Sales Challan module handles the creation and status lifecycle of purchase orders:

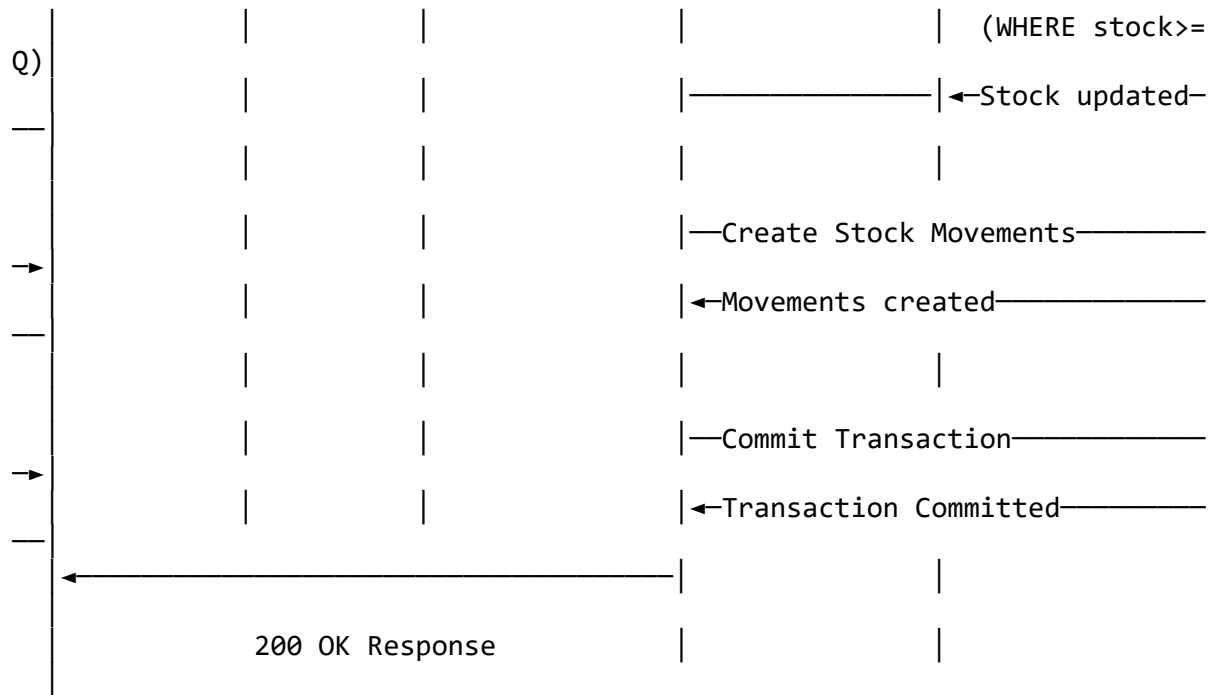


- **DRAFT:** Challans start in this status. Items and quantities can be updated freely, and no inventory is deducted.
- **CONFIRMED:** Deducts stock from inventory and logs a STOCK OUT entry. Once confirmed, a challan is locked and cannot be edited.
- **CANCELLED:** Can be applied to either DRAFT or CONFIRMED challans. Cancelling a confirmed challan restores the original quantities to inventory and logs corresponding STOCK IN entries.

13. Atomic Challan Confirmation

Atomic confirmation ensures that inventory quantities remain accurate when multiple sales agents submit orders at the same time.





Concurrency Lock Mechanics

The server-side implementation uses raw SQL statements with explicit filter conditions inside a database transaction: 1. **Compare-And-Swap (CAS) Challan**

Status: `sql UPDATE "sales_challans" SET "status" = 'CONFIRMED'::"ChallanStatus", "confirmedAt" = NOW() WHERE "id" = ${challanId} AND "status" = 'DRAFT'::"ChallanStatus"` If another request has already updated the status, this query updates 0 rows, prompting the system to rollback and abort. 2. **Conditional Stock Update:** `sql UPDATE "products" SET "currentStock" = "currentStock" - ${quantity}, "updatedAt" = NOW() WHERE "id" = ${productId} AND "currentStock" >= ${quantity} AND "isActive" = true` The database acquires an exclusive row-level write lock on the product table. If the stock level has changed and no longer meets the condition, the query updates 0 rows, and the transaction is aborted.

Running these checks at the query level allows the system to operate safely at the default Read Committed isolation level, avoiding the need for expensive retry loops or serializable transaction rollbacks.

14. API Documentation

The API endpoints are grouped by module. All routes except `/api/auth/login` and `/api/health` require a JWT token in the request header.

Method	Endpoint	Allowed Roles	Purpose / Description
POST	<code>/api/auth/login</code>	Public (All)	Log in and retrieve JWT token
GET	<code>/api/auth/me</code>	All Roles	Retrieve logged-in user profile
GET	<code>/api/customers</code>	All Roles	Retrieve list of customers (supports search and pagination)
GET	<code>/api/customers/:id</code>	All Roles	Retrieve single customer details by ID
POST	<code>/api/customers</code>	ADMIN, SALES	Create a new customer profile
PUT	<code>/api/customers/:id</code>	ADMIN, SALES	Update customer profile details
DELETE	<code>/api/customers/:id</code>	ADMIN	Soft delete customer record
GET	<code>/api/customers/:id/notes</code>	All Roles	List follow-up notes for a customer
POST	<code>/api/customers/:id/notes</code>	ADMIN, SALES	Add a new note to a customer timeline
PUT	<code>/api/customers/:id/notes/:nId</code>	ADMIN, SALES	Edit content of a follow-up note
DELETE	<code>/api/customers/:id/notes/:nId</code>	ADMIN, SALES	Delete a note (Sales can only delete their own)
GET	<code>/api/products</code>	All Roles	Retrieve product catalog list
GET	<code>/api/products/:id</code>	All Roles	Retrieve details of a product by ID
POST	<code>/api/products</code>	ADMIN, WAREHOUSE	Create a new product in the

Method	Endpoint	Allowed Roles	Purpose / Description
PUT	/api/products/:id	ADMIN, WAREHOUSE	Update product catalog
DELETE	/api/products/:id	ADMIN	Soft deactivate a product
POST	/api/inventory/stock-in	ADMIN, WAREHOUSE	Adjust stock level upward (logs movement)
POST	/api/inventory/stock-out	ADMIN, WAREHOUSE	Adjust stock level downward (logs movement)
GET	/api/inventory/movements	All Roles	View entire stock movement ledger
GET	/api/inventory/products/:pId/movements	All Roles	View stock movements for a specific product
GET	/api/challans	ADMIN, SALES	List sales challans (supports pagination and status filter)
GET	/api/challans/:id	ADMIN, SALES	View challan details and item list
POST	/api/challans	ADMIN, SALES	Create a new sales challan (Draft)
PUT	/api/challans/:id	ADMIN, SALES	Update challan item quantities (Draft only)
POST	/api/challans/:id/confirm	ADMIN, SALES	Trigger atomic stock deduction and confirm challan
POST	/api/challans/:id/cancel	ADMIN, SALES	Cancel draft or restore stock from confirmed challan
GET	/api/health	Public (All)	Check system and database health status

15. Frontend Client Application

The frontend is built as a single-page application (SPA) using React, Vite, and Tailwind CSS. * **Component Layout:** The app uses `MainLayout` to manage the navigation sidebar. A helper component, `ProtectedRoute`, handles auth state checks and redirects unauthorized users. * **Navigation Architecture:** The layout maps routes to user roles. The sidebar filters out links that the user does not have permission to access. * **State Management:** Authentication state is managed via React Context (`AuthContext`). API calls are routed through an Axios client that automatically attaches the JWT token from `localStorage` to all requests.

16. Security Infrastructure

Security is implemented across multiple layers of the application stack:

- * **Hashed Passwords:** User passwords are encrypted using bcrypt with 10 salt rounds before being stored. Plain text passwords are never stored or transmitted.
- * **Stateless Tokens:** API requests are authenticated using HS256-signed JWTs, ensuring the server does not need to store session states.
- * **Role Check Enforcements:** Endpoint protection is handled by backend routing rules rather than just hiding UI components in the browser.
- * **Input Validation:** Payloads are parsed and validated by Zod schemas on the server, blocking malformed requests before they hit the database.
- * **Parameterized Queries:** Prisma translates database requests into parameterized queries, protecting the database against SQL injection attacks.
- * **Sensitive Data Scrubbing:** Database queries exclude sensitive columns (like password hashes) before sending JSON responses back to the client.

17. Testing & Verification

The codebase undergoes static code checks and runtime verification to ensure operational stability:

1. **Static Analysis & Builds:**
 - * Backend typecheck runs `npx tsc --noEmit` and build runs `tsc`.
 - * Frontend validation runs `npx tsc --noEmit && npm run build` (bundling via Vite).
 - * Prisma schema is verified using `npx prisma validate`.
2. **API Verification:** Tested using the Postman collection. Key scenarios validated include:
 - * Correct JWT token verification on protected routes.
 - * Proper 403 Forbidden response when users attempt unauthorized operations.
 - * Aborted stock operations when available inventory is insufficient.
 - * Transaction rollbacks when database update operations fail.

18. Deployment Details

The application is deployed on the Render platform.

- * **Database:** Managed PostgreSQL instance (fundsroom-db).
- * **Backend:** Deployed as a web service using the Node environment. The build command compiles the TypeScript source code (`npm install --include=dev && npx prisma generate && npm run build`), and the startup command runs migrations and seeds default test users (`npx prisma migrate deploy && npx prisma db seed && npm start`).
- * **Frontend:** Hosted as a static site. The build output is configured to rewrite all wildcard paths (`/*`) to `/index.html` to support client-side routing.
- * **Environment Configuration:** Variables are injected into the environment by Render to secure database connection strings and JWT signing secrets.

19. User Roles & Permission Matrix

The permission matrix below summarizes the actions allowed for each user role in the application:

Module / Operation	ADMIN	SALES	WAREHOUSE	ACCOUNTS
Authentication & Profile				
User Login / View Profile	Yes	Yes	Yes	Yes
Customer CRM				
View Customer Directory	Yes	Yes	Yes	Yes
Create / Edit Customer Profile	Yes	Yes	No	No
Soft Delete Customer	Yes	No	No	No
Add / Edit Customer Note	Yes	Yes	No	No
Delete Customer Note	Yes (All)	Yes (Own Only)	No	No
Product Catalog				
View Product Catalog	Yes	Yes	Yes	Yes
Create / Edit Product	Yes	No	Yes	No
Soft Deactivate Product	Yes	No	No	No
Inventory Stock Management				
Adjust Stock IN / OUT	Yes	No	Yes	No
View Stock Movements	Yes	Yes	Yes	Yes
Sales Challans				
View Challan List	Yes	Yes	No	No
Create / Update Challan (Draft)	Yes	Yes	No	No
Confirm Challan	Yes	Yes	No	No
Cancel Challan	Yes	Yes	No	No

20. User Guide

Employees use the portal according to their assigned roles: * **ADMIN**: Accesses the dashboard to monitor system activity, manages user accounts, updates product catalogs, and has full read/write permissions across all modules. * **SALES**: Focuses on managing customer relationships, updating CRM details, drafting sales challans, and confirming order checkout when items are ready to ship. * **WAREHOUSE**: Monitors stock alerts, manages incoming shipments (Stock IN) and outgoing adjustments

(Stock OUT), and adds new products to the catalog. * **ACCOUNTS**: Holds read-only access to verify transactions, review challans, and inspect inventory history for auditing purposes.

21. Deployment & Operational Notes

To set up and run the application locally, follow these steps: 1. **System Prerequisites:** Install Node.js (version 18 or higher) and PostgreSQL (version 14 or higher). 2.

Environment Configuration: Create `.env` files in both the backend and frontend directories using the provided `.env.example` templates. 3. **Database Migration:** Run

the Prisma migration tool to set up the database tables: `bash npx prisma migrate deploy` 4. **Seed Database:** Populate the database with default test accounts:

`bash npx prisma db seed` *Admin User:* admin@fundsroom.com / Admin@123

Sales User: sales@fundsroom.com / Sales@123

Warehouse User: warehouse@fundsroom.com / Warehouse@123

Accounts User: accounts@fundsroom.com / Accounts@123

5. **Start Application:** Run the development servers to launch the portal: * Backend:

`npm run dev` (starts on port 5000) * Frontend: `npm run dev` (starts on port 5173)

22. Limitations & Future Scope

The current version of the portal has a few design limitations: * **Token Expiration:**

JWT sessions are stateless and expire after the duration set in `JWT_EXPIRES_IN`. The

system does not support sliding sessions or refresh tokens. * **File Uploads:** The

product catalog only supports text-based descriptions and location tags; it does not

support uploading product images. * **Invoice Export:** Challans are managed within

the web interface, but the system does not support exporting orders as PDF

documents. * **Dashboard Metrics:** The dashboard relies on multiple paginated API

calls to aggregate data. A unified stats endpoint would improve performance.

23. Conclusion

The **Fundsroom Mini ERP + CRM Operations Portal** provides a reliable, secure, and performant platform for managing wholesale operations. Its core feature—atomic challan confirmation—uses database-level updates to prevent race conditions and maintain stock accuracy. Combined with role-based access control, detailed transaction logs, and a responsive React interface, the portal ensures consistent and secure business workflows.